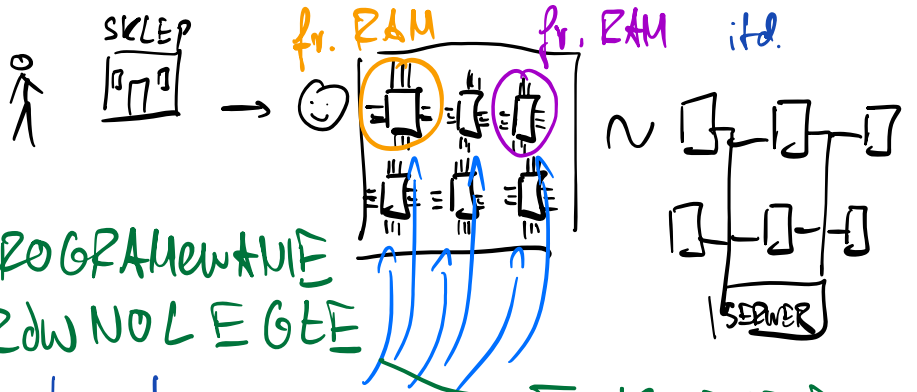
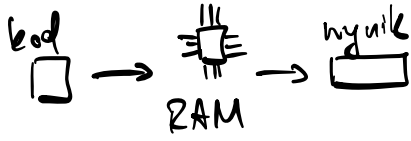


CEL: „zrozumieć”
kodu w Pythonie



PROGRAMOWANIE RÓWNOLEGŁE

STANDARDOWO: ←



jest oparty w bitach:

~ 1 | interpretacja
~ 0 | matematyczna

MODEL

p	q	p ^ q
1	1	0
1	0	1
0	1	1
0	0	0

przeds ← 1 → przyszłość
false ← 0 → nie przyszłość

DANE ~ liczby

systemy liczbowe
↓
dziesiętne dwójkowe

↓
dla każdego „długiego”
zadania na „mniejszych”
↓
CPU Vs GPU
Pogromcy Mitów

III Kwestie:

1. Co w operacji
dla tego komputer
+ optymalizacja
2. Jak realnie wygląda to w kodzie?

3.

nie wszyscy
wzrosty
interakcje punktów „graczami”,
czyli o. kodu interakcyjnego
równoległe / jednoczesne

OPERACJE

Obciążenie dla
procesora

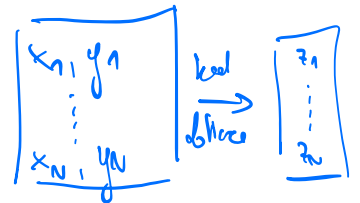
CPU-bound
sprowadza się
bezpośrednio koszt
obliczeniowy
 $x \cdot y = z$

for para in lista:
z = para[0] * para[1]
wynik.append(z)

PROCESY

I/O-bound
„input”
„output”

sprowadza się pobieranie
; zwracanie danych
pobieramy x, y
wypisujemy z

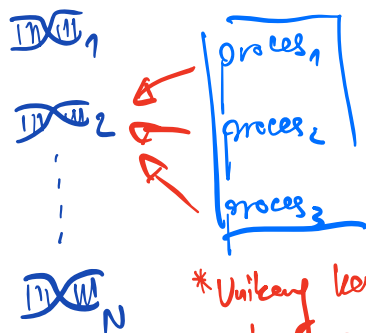


WĄTKI

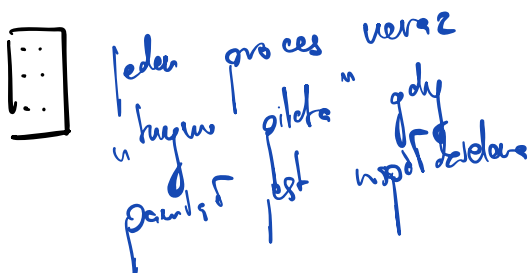
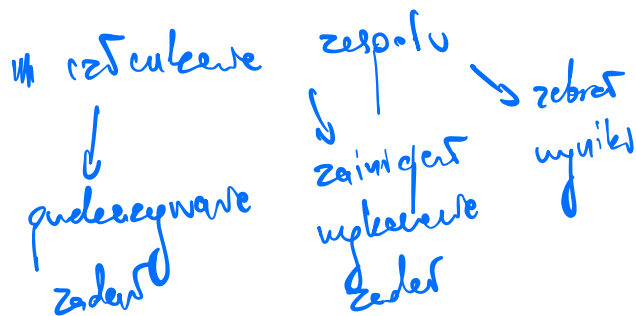
(multi processing)

(multi threading)

I PROCESSY



* Unikum konstanty
z tego samego
długości oraz
długości w różnych procesach



II WĄTKI



VS WÓZ

race condition *

Tu należy mieć również

GIL

code Python

Python → interpreter → kompilacja

wie wiele wątków
rezerwacji, przydziel
oni zwrócić

* Zdarza się wątek (zawieszony wątek)
w wątku (bez efektu)



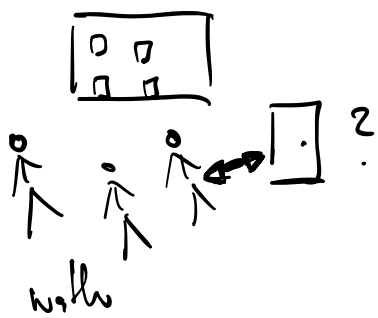
GIL nie może
zaobserwować

Jakie problemy może być GIL?

nieprofesjonalne
wymagania
biblioteki

zawieszony wątek
jak utrudniać
się śledzić

funkcjonalne z uwagi
wykresy



1. Podstawy procesy : with
2. Mutex, blokada, bariery, flagi : zewnętrzne mechanizmy
3. Race condition *only pythoed*
4. Murowane macierzy.

$$\begin{bmatrix} \dots \\ \dots \\ \dots \end{bmatrix} \cdot \begin{bmatrix} \dots \\ \dots \\ \dots \end{bmatrix} = ?$$

Uporządkujmy rzeczniczko:

inne implementacje

dwa czytelnosć

wysokość

→ dalsze od tego, jakie
rozumiemy komputer

język programowania

interpretacyjny

for - in range(...):

(1) [... = ...]
[...()] (2)
[print(...)]

Tomaz (Interpreter)

[01101010] → bajt

CPython

wiele elementów
wpisywać w C

KOD BINARNY
(KOD MASZYNOWY)

01101010 ...

↓
rozumiemy wykonywany
przez komputer

dopiero wie
dobra do bzdur,
kod "dziw",
jest wykonywany

{ kod bajtowy } .pyc

01101010 ... → wykonywane (1)
10011010 ... → wykonywane (2)

["niekolejny" język] ?
kompilacja

Flumaczowe linii → wykonywane linii X linia linii

→
wzrostu kodu
bezporednio w IDE
ze skrypty lub zrygły

→
wzrostu kodu
bez zrygnowo czasu
w kompilowane

$\begin{bmatrix} \dots \\ \dots \\ \dots \end{bmatrix}$ kod w C

↓
Cofos jest Flumaczowe

$\begin{bmatrix} 01101010 \\ 10011010 \\ \dots \end{bmatrix}$

.exe

ALE

kompilacyjny kod

małe dzieła
szybkie

duży kompilator
weryfikacja kodu przed uruchomieniem

delikatne jest bardzo ciężkie

Podsumowanie:

- kompilator tłumaczy cały kod raz
- interpreter tłumaczy: wykonywa kod linijka po linijce
- CPython to domyślny interpreter Python
- język programowania może być zarówno kompilowany, jak i interpretowany
Zależy to od implementacji.